Αναφορά Εργασίας: Συγχρονισμός Διεργασιών

1. Εισαγωγή
Στόχος της εργασίας ήταν η υλοποίηση των προγραμμάτων multiproc_1 και multiproc_2, που επιδεικνύουν τον συγχρονισμό παράλληλων διεργασιών με χρήση POSIX named semaphores, ώστε να αποφευχθούν race conditions κατά την εκτύπωση.

2. multiproc_1 - Αμοιβαίος Αποκλεισμός στην Εκτύπωση
Το πρόγραμμα δέχεται έναν ακέραιο rep και ένα ή περισσότερα strings. Δημιουργεί τόσες διεργασίες-παιδιά όσα τα strings, και κάθε παιδί εκτυπώνει το αντίστοιχο string rep φορές μέσω της display().

Πρόβλημα που αντιμετώπισα: Χωρίς συγχρονισμό, οι χαρακτήρες από διαφορετικές διεργασίες αναμειγνύονταν, γιατί η display() γράφει έναν χαρακτήρα τη φορά με usleep(100) μεταξύ τους.

Πρόβλημα με sem_open: Αρχικά όταν έτρεχα το πρόγραμμα δύο φορές γρήγορα, η sem_open() απέτυχε με σφάλμα EEXIST γιατί το semaphore από την προηγούμενη εκτέλεση παρέμενε στο σύστημα. Το έλυσα καλώντας sem_unlink() πριν τη sem_open(), ώστε να σβήνεται τυχόν παλιό semaphore με το ίδιο όνομα.

Λύση συγχρονισμού: Χρησιμοποίησα ένα binary named semaphore (αρχική τιμή 1) ως mutex. Κάθε παιδί κάνει sem_wait() πριν την display() και sem_post() μετά, εξασφαλίζοντας αποκλειστική πρόσβαση.

Επιπλέον πρόβλημα: Ακόμα και με το semaphore παρατηρούσα interleaving. Ο λόγος ήταν ότι η display() χρησιμοποιεί συνδυασμό printf/putchar (buffered) και write() (unbuffered). Ο χαρακτήρας \n έμενε στο buffer και εκτυπωνόταν αφού η επόμενη διεργασία είχε ήδη αρχίσει. Το έλυσα προσθέτοντας fflush(stdout) μεταξύ display() και sem_post().

Πρόβλημα με wait(): Αρχικά δεν είχα βάλει wait() στην πατρική διεργασία, με αποτέλεσμα το πρόγραμμα να τερματίζει πριν προλάβουν τα παιδιά να εκτυπώσουν όλες τις γραμμές τους. Το διόρθωσα προσθέτοντας loop με wait(NULL) για κάθε παιδί.

3. multiproc_2 - Barrier Συγχρονισμός
Το multiproc_2 προσθέτει την απαίτηση ότι κάθε παιδί πρέπει να καλέσει init() πριν τις εκτυπώσεις, και καμία διεργασία δεν ξεκινά εκτύπωση αν δεν έχουν τελειώσει όλες το init(). Αυτό είναι κλασικό πρόβλημα barrier synchronization.

Χρησιμοποίησα τρία named semaphores και ένα POSIX shared memory segment:

sem_display (αρχ. τιμή 1): mutex για αποκλειστική πρόσβαση στη display(), ίδια λογική με το multiproc_1.

sem_cnt_mtx (αρχ. τιμή 1): mutex που προστατεύει τον μετρητή finished_count στο shared memory. Κάθε παιδί τον αυξάνει κατά 1 αφού τελειώσει το init().

sem_barrier (αρχ. τιμή 0): το φράγμα. Κάθε παιδί κάνει sem_wait() αφού ενημερώσει τον μετρητή. Το τελευταίο παιδί (αυτό που βρίσκει finished_count == num_strings) κάνει N φορές sem_post(), ξυπνώντας όλες τις αναμένουσες διεργασίες ταυτόχρονα.

Γιατί shared memory: Μετά το fork() κάθε παιδί έχει δικό του αντίγραφο της μνήμης, οπότε ο μετρητής δεν μπορεί να είναι απλή μεταβλητή της πατρικής. Χρησιμοποίησα shm_open() και mmap() με MAP_SHARED για να μοιραστεί ο μετρητής μεταξύ όλων των διεργασιών.

4. Προβλήματα και Συμπεράσματα

Αρχικά δοκίμασα να χρησιμοποιήσω sem_getvalue() για τον έλεγχο του μετρητή, αλλά αυτή δεν είναι ατομική με το sem_post() — υπάρχει χρονικό παράθυρο όπου δύο διεργασίες μπορούν να διαβάσουν την ίδια τιμή. Η λύση ήταν το ξεχωριστό mutex (sem_cnt_mtx) για atomic read-modify-write του finished_count.

Η εργασία με βοήθησε να κατανοήσω καλύτερα πώς συγχρονίζονται διεργασίες με semaphores και πώς μοιράζονται δεδομένα μέσω shared memory σε Unix.